FPGA-Based Adaptive Beamformer for 5G NR

Project Update Report

Date: April 24, 2026 — Phases 1.1 & 1.2 Complete — Phase 1.3 Starting

Executive Summary

This report documents the completion of Phase 1.1 and Phase 1.2 of the FPGA-Based Adaptive Beamformer project. The project implements a beamforming receiver for a 5G New Radio (NR) base station on an FPGA, targeting an 8-element Uniform Linear Array (ULA) with a conventional delay-and-sum beamformer in Phase 1 and an adaptive LMS beamformer in Phase 2.

Both MATLAB simulation stages have been completed and verified. The floating-point golden reference and the fixed-point analysis are consistent, word lengths are locked, and RTL test vectors have been exported. The project is ready to proceed to Phase 1.3 (SystemVerilog RTL implementation).

Project Plan

The project is structured into two phases, each with five sub-stages. Phase 1 delivers a non-adaptive beamformer with fixed weights stored in a ROM. Phase 2 replaces the ROM with an adaptive LMS weight-update engine, turning the system into a self-steering beamformer that requires no prior knowledge of the target direction at runtime.

Phase 1 — Non-Adaptive Beamformer

1.1. Algorithm simulation (MATLAB, floating-point) — Establish the golden reference. Validate the delay-and-sum beamformer analytically: beam direction, array gain, SINR improvement. This is the root of the verification chain. **[Complete]**

1.2. Fixed-point analysis (MATLAB) — Determine the minimum word lengths for inputs, weights, multiplier outputs, and the accumulator. Confirm that quantisation does not degrade SINR beyond acceptable limits. Export test vectors for the RTL testbench. **[Complete]**

1.3. RTL implementation (SystemVerilog) — Write five hand-crafted modules: a pipelined complex multiplier, a parallel accumulator, a weight ROM, a top-level wrapper, and a testbench. All widths are parameterised; DSP48 inference is the synthesis target. **[Next]**

1.4. Co-simulation and verification — Drive the RTL with the exported test vectors. Every output sample must match the fixed-point MATLAB reference to within one LSB. Directed edge-case tests (impulse, saturation, all-zero input) are also run.

1.5. Synthesis and timing closure (Vivado) — Synthesise for Artix-7 / Zynq-7000 targeting 200 MHz. Verify DSP48 inference, extract resource and timing reports.

Phase 2 — Adaptive LMS Beamformer

2.1. LMS algorithm simulation (MATLAB) — Validate that the LMS weight-update loop converges from an arbitrary starting point using only a pilot reference signal.

2.2. Fixed-point LMS analysis (MATLAB) — Verify convergence with finite-precision arithmetic. Determine word lengths for the weight accumulator, error signal, and step size.

2.3. HLS implementation (Vitis HLS) — Describe the LMS update in C++ with `ap_fixed` types and HLS pragmas. Co-simulate against the MATLAB reference and export packaged RTL.

2.4. Integration and verification — Replace the Phase 1 weight ROM with a writeable register bank. Add the HLS LMS engine and a pilot-sequencer FSM. Verify weight convergence and output SINR improvement.

2.5. Synthesis and comparison — Synthesise the full adaptive system. Document the additional resource cost and power delta compared to Phase 1.

System Parameters

Parameter	Value
Array type	Uniform Linear Array (ULA)
Number of elements M	8
Element spacing	$d/\lambda = 0.5$ (half-wavelength)
Target signal angle	30° from broadside
Interferer angle	−20° from broadside
Input SNR	10 dB
Input SIR	0 dB (equal power)
Time samples	512
Random seed	42 (fixed for reproducibility)
Target FPGA	Artix-7 / Zynq-7000 (baseline)
Target clock	200 MHz

Table 1: Simulation and implementation parameters.

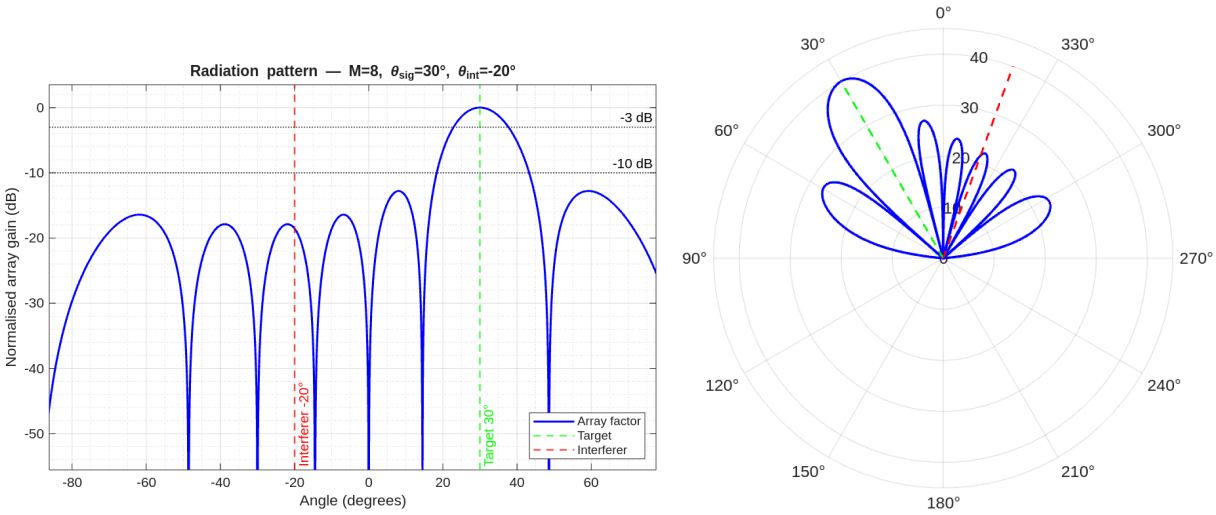
Phase 1.1 — Floating-Point Simulation

The conventional delay-and-sum beamformer was implemented in MATLAB. Weights are the normalised steering vector toward the target:

$$\mathbf{w} = \frac{\mathbf{a}(\theta_{\text{sig}})}{\|\mathbf{a}(\theta_{\text{sig}})\|}, \qquad y(n) = \mathbf{w}^H \mathbf{x}(n)$$

Metric	Value	Notes
Array gain (signal)	+9.03 dB	$\approx 10\log_{10}(8)$ ✓
Array gain (interferer)	−9.53 dB	Suppression at −20°
Input SINR	−2.55 dB	Before beamforming
Output SINR	+15.78 dB	After beamforming
SINR improvement	18.33 dB	Golden reference

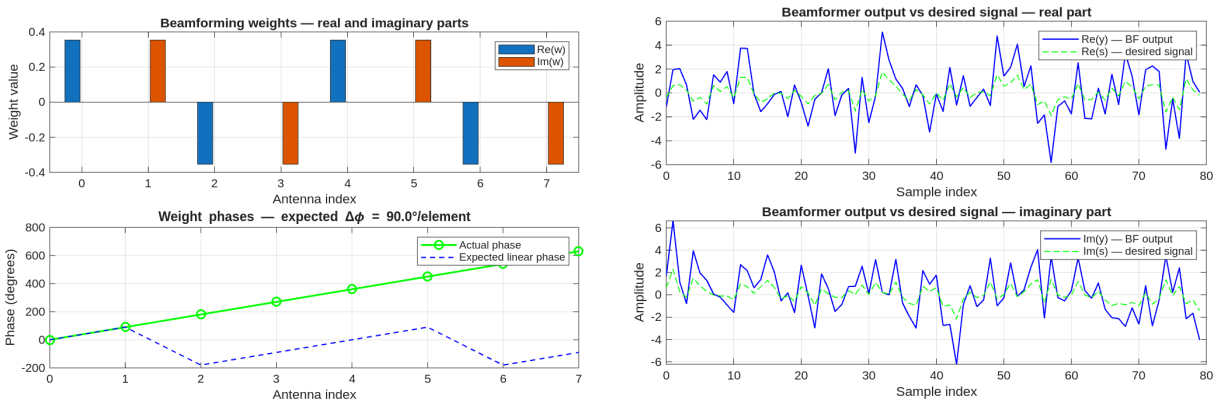
Table 2: Phase 1.1 verified metrics.



(a) Radiation pattern (rectangular). Main lobe at 30°, null near the interferer at −20°.

(b) Polar radiation pattern showing the directional beam.

Figure 1: Phase 1.1 — Array radiation patterns.



(a) Weight values and phases. Linear phase progression of 90°/element confirms correct steering.

(b) Beamformer output vs desired signal (first 80 samples).

Figure 2: Phase 1.1 — Weights and time-domain output.

Phase 1.2 — Fixed-Point Analysis

Locked-In Word Lengths

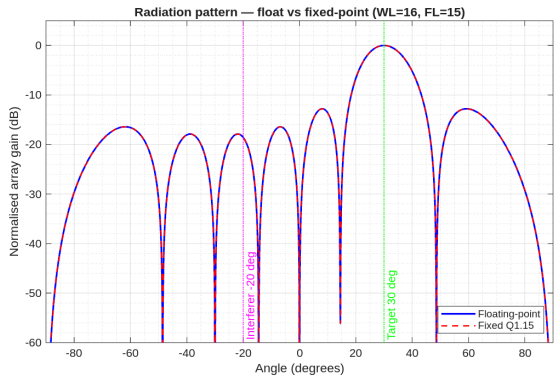
Signal	Format	WL	FL	Range
Input samples $x(n)$	Q1.15	16	15	$[-1, +1)$
Weights $w(m)$	Q1.15	16	15	$[-1, +1)$
Multiplier output	Q2.30	32	30	$[-2, +2)$
Accumulator	Q4.31	36	31	$[-16, +16)$
Beamformer output $y(n)$	Q1.14	16	14	$[-2, +2)$

Table 3: Locked-in word lengths for the Phase 1 RTL pipeline.

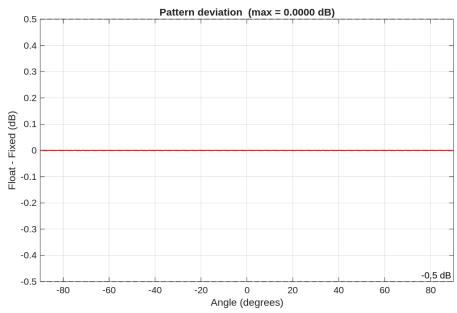
Verification Results

Metric	Result	Tolerance	Status
Output SINR degradation	0.00 dB	—	**PASS**
Peak pattern deviation	0.0000 dB	< 0.5 dB	**PASS**
Null depth deviation	0.0000 dB	< 5.0 dB	**PASS**
3 dB beamwidth deviation	0.0000°	—	**PASS**
Output SQNR	33.51 dB	—	**PASS**

Table 4: Phase 1.2 verification results against Phase 1.1 golden reference.

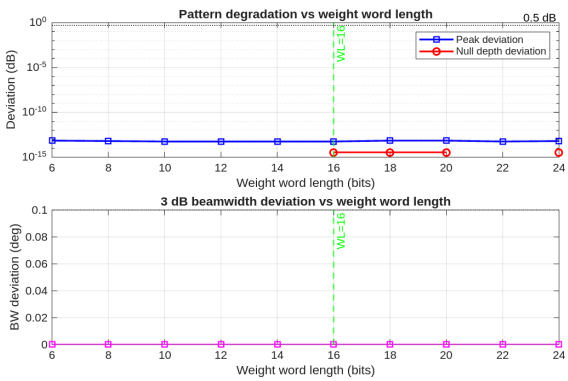


(a) Float vs fixed-point radiation pattern. The red dashed curve is completely hidden under the blue — patterns are indistinguishable at Q1.15.

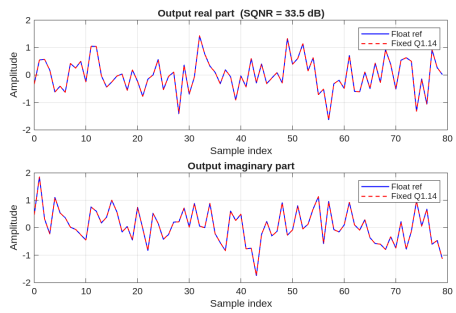


(b) Point-by-point pattern deviation. Spikes at deep nulls are a log-scale artefact; all significant angles are within ±machine epsilon.

Figure 3: Phase 1.2 — Radiation pattern comparison.



(a) Pattern degradation vs weight word length. Deviations are at $\sim 10^{-14}$ dB for all word lengths tested (6–24 bits), confirming that 16 bits is well past the degradation knee.



(b) Float vs fixed-point beamformer output (first 80 samples). SQNR = 33.51 dB; fixed-point output tracks float reference closely.

Figure 4: Phase 1.2 — Word-length sweep and output comparison.

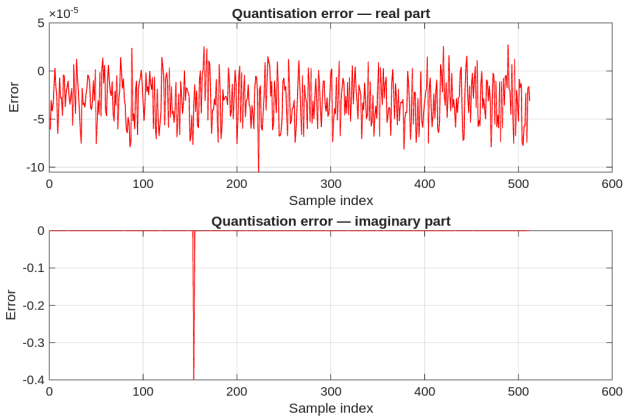


Figure 5: Phase 1.2 — Quantisation error (float minus fixed-point output). Error is small, zero-mean, and uncorrelated with the signal — consistent with benign quantisation noise.

Exported Test Vectors

File	Lines	Contents
`vectors/weights.hex`	16	8 complex weights, Re then Im, Q1.15
`vectors/inputs.hex`	8 192	$8 \times 512 \times 2$ samples, Q1.15
`vectors/expected_output.hex`	1 024	512 complex output samples, Q1.14

Table 5: Test vector files for `$readmemh` in the Phase 1.4 testbench.

Next Steps — Phase 1.3 RTL Implementation

Five SystemVerilog modules to be written in order:

1. `complex_mult.sv` — Two-stage pipelined complex multiplier. Four real multiplies, two pipeline registers. Parameterised widths for DSP48 inference.
2. `complex_accumulator.sv` — Sums $M = 8$ complex products per sample clock using a pipelined adder tree.
3. `weight_rom.sv` — Initialised from `weights.hex`, delivers all M weights in parallel each clock.
4. `beamformer_top.sv` — Instantiates M multipliers, one accumulator, and the weight ROM. Manages pipeline-depth alignment and valid-signal propagation.
5. `beamformer_tb.sv` — Reads `inputs.hex` and `expected_output.hex`, drives the DUT, compares output sample-by-sample, reports pass/fail.

Acceptance criterion: all 512 output samples match `expected_output.hex` to within ± 1 LSB.

Generated automatically at end of Phase 1.2. For full design rationale see `DESIGN.md`.